

Module 1: Lesson 2

MLP for classification: Timing AAPL stock



Outline

- ▶ MLP for classification
- ▶ Binary cross-entropy loss function
- ▶ Class weights
- ▶ Model predictive performance
- ▶ Trading strategy performance

MLP for classification

In the last module of Machine Learning, we dealt with an example of how to using neural networks, specifically MLPs, to time the momentum factor. Arguably, we used a somewhat simple model that did not really deliver an stellar performance when it came to predicting the forward-looking return of the factor.

Now, we'll insist on this challenging task with some changes:

- ▶ Perform a **classification** task: instead of focusing on predicting the forward-looking return of a financial series, we will merely try to predict whether this return will be positive ($y_i = 1$) or negative ($y_i = 0$).
- ▶ Focus on a **single stock (AAPL)**. Rather than looking at a factor that explains the cross-section of returns, we will limit our investigation to a single stock time series.
- ▶ Incorporate new features such as **dropout** and **class weights**.

Before jumping to a road map on the implementation you will see in detail in the notebook, a few caveats:

1. Focus on a single stock (AAPL). The model may not work as well/badly for others (feel free to try!)
2. Focus on a specific time period (1980–2022). The model may not work as well/badly for others (or future!).

Building our model (1/2)

We'll build our model in a very similar way to what we have done before:

- ▶ **Layers:** 3 hidden layers with 25, 15, and 10 units respectively + output layer
- ▶ **Dropout:** dropout layer of 20% after each hidden layer
- ▶ **Optimizer & Learning rate:** Adam & 10^{-6}
- ▶ **Activation function:** ReLU for all hidden layers + Sigmoid for output layer

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 25)	150
dropout (Dropout)	(None, 25)	0
dense_1 (Dense)	(None, 15)	390
dropout_1 (Dropout)	(None, 15)	0
dense_2 (Dense)	(None, 10)	160
dropout_2 (Dropout)	(None, 10)	0
dense_3 (Dense)	(None, 1)	11
=====		
Total params: 711		
Trainable params: 711		
Non-trainable params: 0		

Building our model (2/2)

More (new!) things about our model:

- **Loss function:** Binary cross-entropy

$$H(y_i) = -\frac{1}{N} \sum_{i=1}^N y_i \log(p(y_i)) + (1 - y_i) \log(1 - p(y_i))$$

- **Metric:** (binary) accuracy to evaluate our model performance

```
import tensorflow as tf

tf.keras.backend.clear_session()
tf.random.set_seed(1234)

act_fun = 'relu'
hp_units = 25
hp_units_2 = 15
hp_units_3 = 10
n_dropout = 0.2

model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(units = hp_units, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = hp_units_2, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = hp_units_3, activation=act_fun))
model.add( tf.keras.layers.Dropout(n_dropout))
model.add(tf.keras.layers.Dense(units = 1, activation='sigmoid'))

hp_lr = 1e-5

adam = tf.keras.optimizers.Adam(lr=hp_lr)

model.compile(optimizer = adam, loss='binary_crossentropy', metrics=['accuracy'])
```

Class weights

One problem that may always be present when we perform classification tasks is the under-representation of a certain class in the sample data. This can cause the model to overlook an important part of the data in the sample while training.

- ▶ For example, suppose we want to predict firm bankruptcy using imperfect historical data. Bankruptcy may not be a very frequent event in our sample but is truly our main variable of interest.
- ▶ To mitigate this problem, we can make the model pay more attention to certain classes. This can be done by introducing **class weights**, which simply consist of directing Keras to pay more attention to a certain class.
- ▶ In our case, we will try to overweight the negative classes in our sample (i.e., those instances where the 120-day-forward return on AAPL is negative).
- ▶ This is actually something we will introduce while training the model, similar to the EarlyStopping callback:

```
from sklearn.utils import class_weight

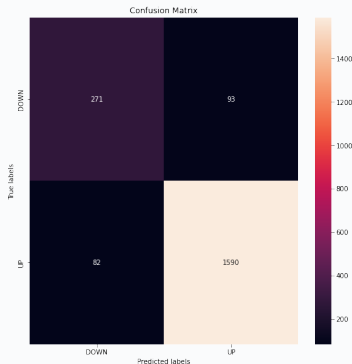
class_weight = {0: (np.mean(y_train)/0.5)*1.2, 1: 1.0}
print(class_weight)

history = model.fit(X_train, y_train, validation_split=0.2, epochs=500, batch_size=32,
                    verbose=2, callbacks=[es], class_weight=class_weight)
```

Model predictive performance

So how does our predictive model perform in the test sample?

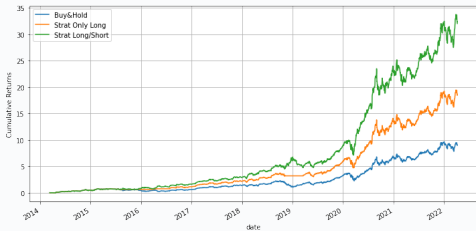
- To assess that, we will come back to an old friend: the confusion matrix!



- Model accuracy in Test: 91.40%

Trading strategy performance

Predictive power of the model in a test sample is important to assess the performance of our model. But if we want to investigate whether we could use this for trading, we better perform a backtest:



Usual **caveats** on backtesting:

- ▶ Specific stock (AAPL)
- ▶ Specific time period for training and testing (1980–2022)
- ▶ Based on historical information (what about things that never happened before?)

Summary of Lesson 2

In Lesson 2, we have:

- ▶ Developed an MLP model for classification to predict whether a certain stock will deliver a positive or negative return in the next few days
- ▶ Enhanced the performance of our MLP neural networks with features such as dropout or class weights.

⇒ **TO DO NEXT:** Go to the associated **Jupyter Notebook** to cover the full content of this lesson.

⇒ **In the next lesson**, we'll keep improving our neural network models by introducing hyperparameter tuning.